

جعبه ابزار گیت: راهنمای کاربردی برای توسعه‌دهندگان حرفه‌ای

۲۹ دستور کلیدی برای تسلط بر گردش کار شما

گردش کار از شروع تا ثبت تاریخچه

هر پروژه با این دستورات آغاز می شود و هر روز با همین چرخه ادامه پیدا می کند. اینها ابزارهای اصلی شما برای ردیابی و مشاهده تغییرات تغییرات در مخزن محلی هستند.

git clone [URL]

یک مخزن را از یک URL به سیستم شما کپی می کند.

git status

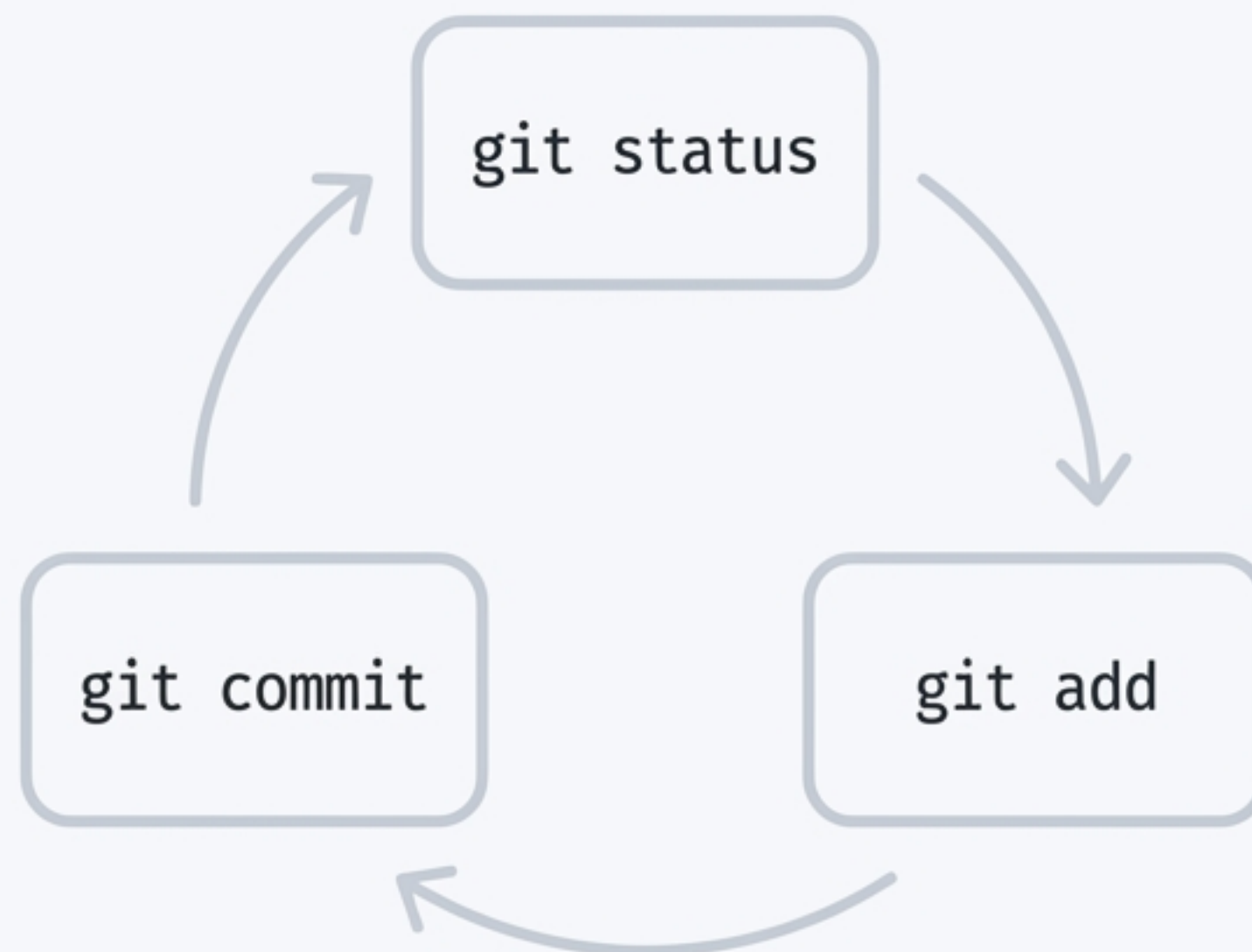
وضعیت فعلی فایل ها را نشان می دهد: تغییر کرده، استیج شده، یا جدید.

git log

لیست تاریخچه کامیت ها را همراه نام نویسنده، پیام، تاریخ و هش نشان می دهد.

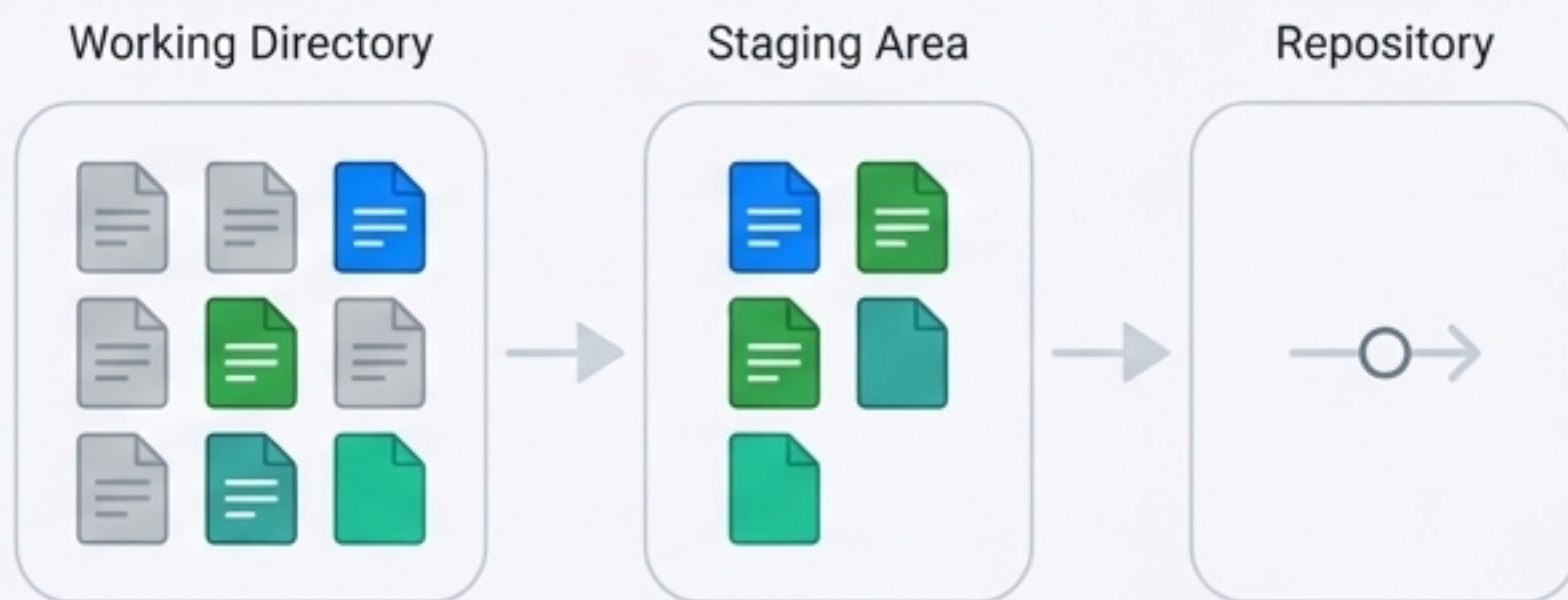
git diff

تفاوت بین فایل های تغییر داده شده و نسخه ی قبلی در مخزن را نمایش می دهد.



ثبت تغییرات: ساختن تاریخچه‌ای معنادار

استیج کردن و کامیت کردن، هنر بسته‌بندی تغییرات در واحدهای منطقی است. هر کامیت باید یک داستان کوچک و کامل را روایت کند.



git add .

تمام فایل‌های تغییر یافته را به استیج اضافه می‌کند.

git add <file>

یک فایل یا پوشه خاص را به استیج اضافه می‌کند.

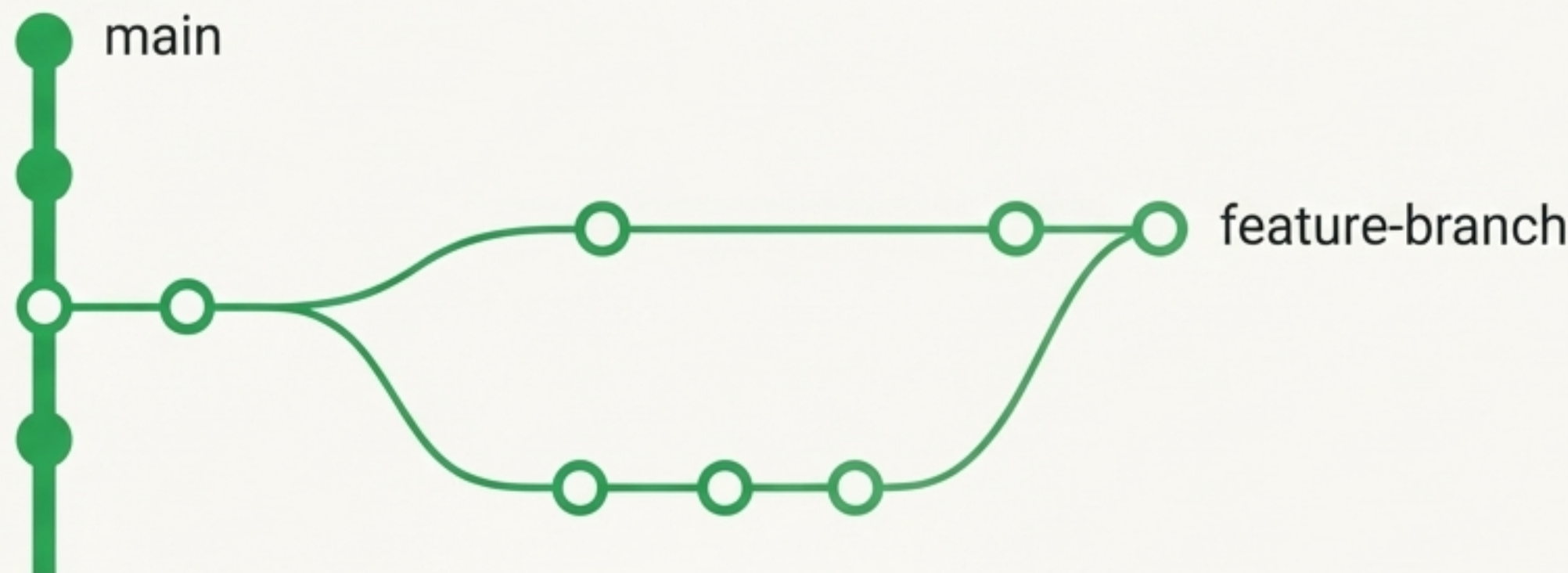
git commit -m "پیام شما"

تغییرات استیج شده را با یک پیام کوتاه کامیت می‌کند.

git commit -am "پیام شما"

فایل‌هایی که قبلاً track شده‌اند را همزمان استیج و کامیت می‌کند (فایل جدید را استیج نمی‌کند).

استراتژی شاخه‌ها: ایزوله‌سازی و توسعه موازی



شاخه‌سازی (Branching) قلب تپنده کار تیمی در گیت است. با ایزوله کردن فیچرها، رفع باگ‌ها و آزمایش‌های جدید، از پایداری شاخه اصلی محافظت کنید.

`git switch <branch-name>`

`git switch -c <new-branch-name>`

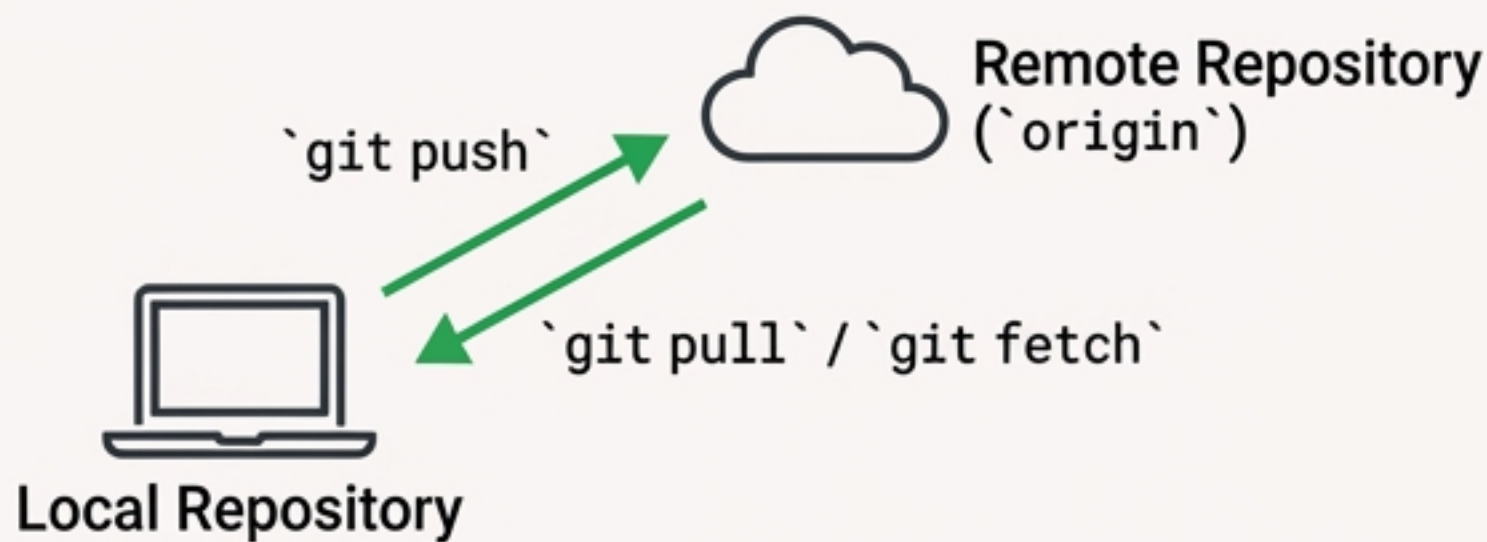
`git branch -D <branch-name>`

به شاخه‌ی مشخص شده جابجا می‌شود.

یک شاخه جدید می‌سازد و بلافاصله به آن سوئیچ می‌کند.

یک شاخه را به صورت اجباری در مخزن محلی حذف می‌کند.

همگام‌سازی با سرور: اشتراک‌گذاری و دریافت تغییرات



مخزن محلی شما باید همواره با منبع اصلی حقیقت (ریموت) در ارتباط باشد. این دستورات به شما کمک می‌کنند تا کار خود را به اشتراک بگذارید و از آخرین تغییرات تیم مطلع شوید.

کامیت‌های محلی را به مخزن ریموت ارسال می‌کند. `git push`

برای اولین بار، شاخه جدید را به ریموت ارسال کرده و upstream را تنظیم می‌کند. `git push -u origin <branch-name>`

آپدیت‌ها را از شاخه main سرور دریافت می‌کند اما با شاخه محلی ادغام (merge) نمی‌کند. `git fetch origin main`

تنها در صورتی تغییرات را دریافت و ادغام می‌کند که تاریخچه کاملاً خطی باشد (fast-forward). این روش از `git pull --ff-only` merge commit های ناخواسته جلوگیری می‌کند.

اصلاحات سریع: بازنویسی و بازنشانی محلی



اشتباهات بخشی از فرآیند توسعه هستند. گیت ابزارهای قدرتمندی برای اصلاح آن‌ها، چه کوچک و چه بزرگ، در اختیار شما قرار می‌دهد.

`git commit -a --amend`

`git checkout --`

`git reset --hard origin/main`

`git reset --hard HEAD~N`

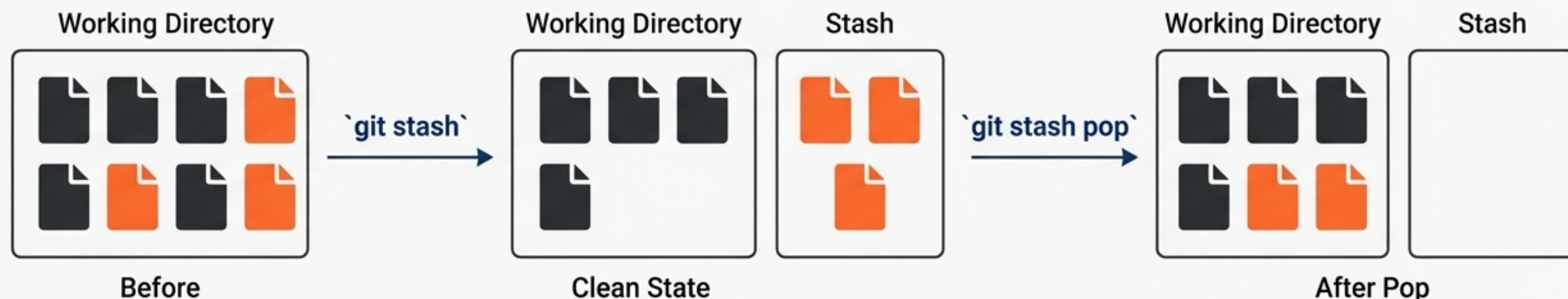
آخرین کامیت را بازنویسی می‌کند (برای تغییر پیام یا افزودن تغییرات فراموش شده).

تمام تغییرات اعمال شده در فایل‌های محلی (work directory) را لغو کرده و به وضعیت آخرین کامیت برمی‌گرداند.

تمام تغییرات و کامیت‌های محلی را حذف کرده و شاخه را دقیقاً به وضعیت شاخه main در ریموت بازمی‌گرداند.

تعداد N کامیت آخر را به‌طور کامل از تاریخچه حذف می‌کند.

جادوی Stash: ذخیره موقت تغییرات



زمانی که نیاز دارید فوراً شاخه را تغییر دهید اما کارتان نیمه‌کاره است، stash به شما اجازه می‌دهد تغییرات را موقتاً کنار بگذارید و فضای کاری خود را تمیز کنید.

تمام تغییرات فعلی (tracked files) را مخفی کرده و فضای کاری را به وضعیت HEAD برمی‌گرداند.
`git stash`
لیست تمام stash‌های ذخیره شده را نمایش می‌دهد.
`git stash list`
آخرین stash را بر روی فضای کاری فعلی اعمال کرده و آن را از لیست حذف می‌کند.
`git stash pop`

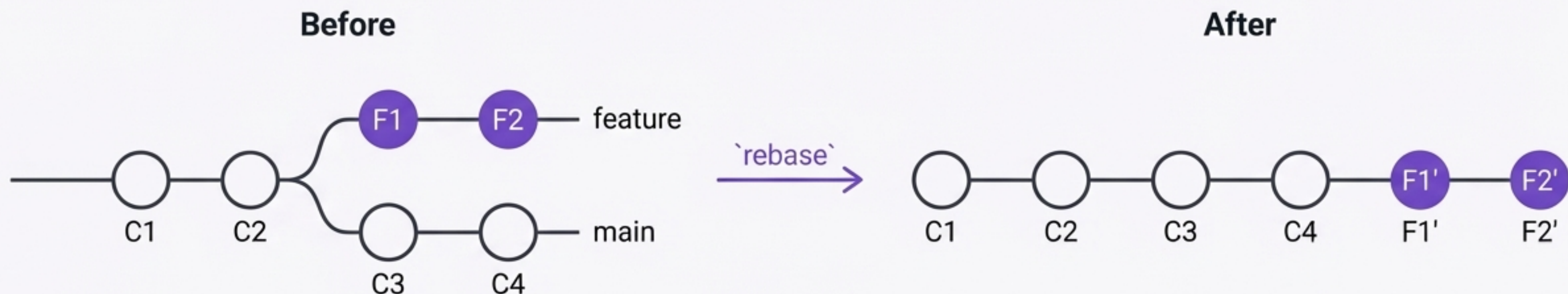
ابزار نهایی بازیابی: `reflog`

این دستور شما را زمانی نجات می‌دهد که فکر می‌کنید همه چیز را از دست داده‌اید. `reflog` تاریخچه کامل تمام حرکات HEAD (مانند checkout، reset، reset، ent، rebase) را ثبت می‌کند و به شما اجازه می‌دهد به هر نقطه‌ای در گذشته بازگردید.

✓	HEAD@{0}: commit: fixed crucial bug
↶	HEAD@{1}: reset: moving to HEAD~2
↶	HEAD@{2}: rebase: finishing interactive rebase
→	HEAD@{3}: checkout: moving from feature-X to develop
✓	HEAD@{4}: commit: updated documentation
↶	HEAD@{5}: reset: moving to origin/main

تاریخچه کامل تمام تغییرات در HEAD را نمایش می‌دهد. این دستور برای بازیابی کامیت‌ها یا شاخه‌های `git reflog` حذف‌شده بسیار مهم و کاربردی است.

قدرت Rebase: خلق تاریخچه‌ای خطی و خوانا



به جای ایجاد merge commit های متعدد، `rebase` به شما اجازه می‌دهد تا زنجیره کامیت‌های خود را بر روی آخرین نسخه شاخه اصلی قرار دهید. نتیجه، یک تاریخچه تمیز و قابل فهم است.

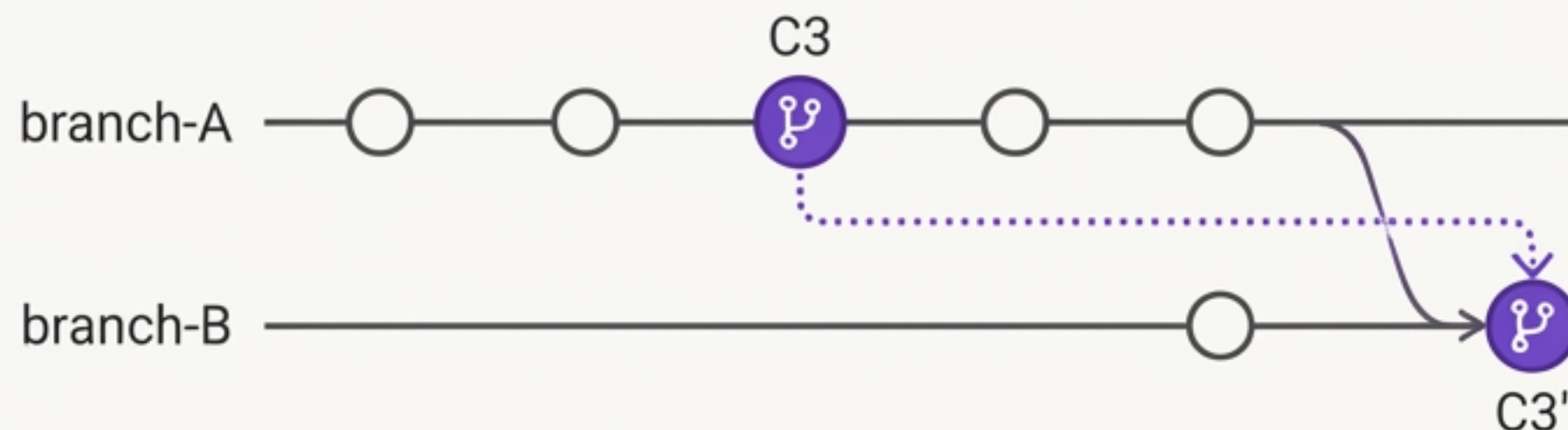
- ری بیس تعاقبی را برای کامیت رآغاز آخرین تلجییت‌های دریسیر کامیت آخیر را موزهد می‌کند: `git rebase origin/main` تاریخچه شاخه فعلی شما را بر روی آخرین تغییرات شاخه `main` بازنویسی می‌کند.

- ری بیس تعاملی را برای N کامیت آخر آغاز می‌کند. این به شما امکان ادغام (squash)، تغییر پیام `git rebase -i HEAD~N` (reword) یا حذف (drop) کامیت‌ها را می‌دهد.



هنر تاریخچه: بازنویسی پیشرفته

عملیات جراحی: انتقال دقیق کامیت‌ها



گاهی نیاز دارید فقط یک کامیت خاص را از شاخه‌ای دیگر به شاخه فعلی خود منتقل کنید، یا دسته‌ای از کامیت‌ها را به نقطه دیگری از تاریخچه جابجا کنید. این دستورات برای این کار ساخته شده‌اند.

- **git cherry-pick <commit-hash>**

یک کامیت مشخص را از هر شاخه‌ای انتخاب کرده و یک کپی از آن را روی شاخه فعلی اعمال می‌کند.

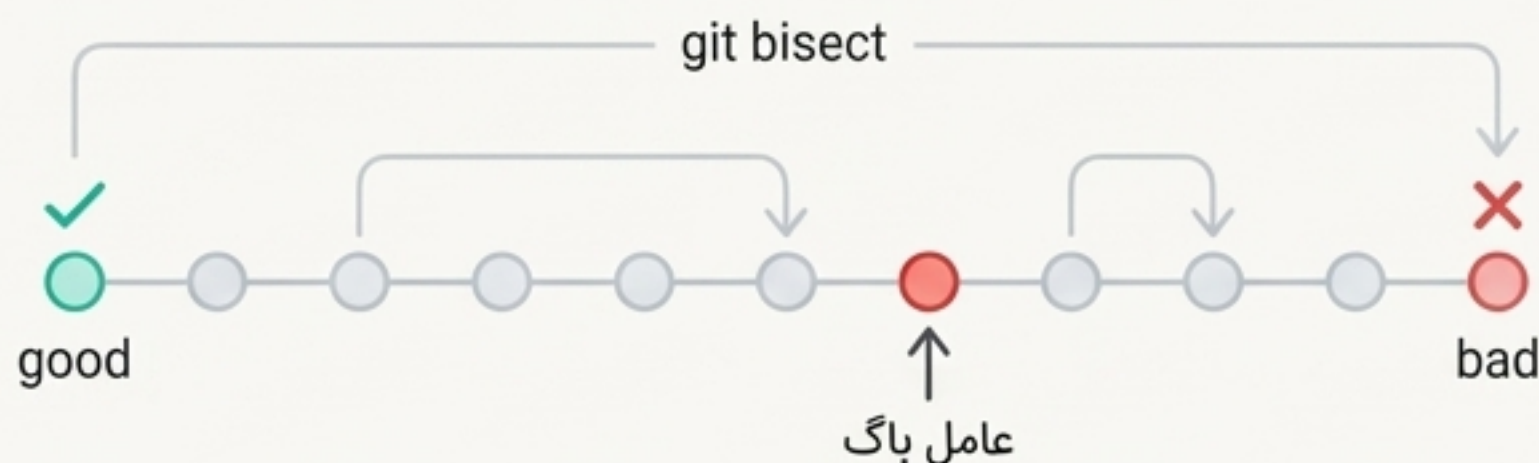
- **git rebase --onto <new-base> <old-base> <branch>**

دسته‌ای از کامیت‌ها را از یک نقطه به نقطه دیگر تاریخچه جابجا می‌کند. (ابزاری بسیار قدرتمند برای بازآرایی‌های پیچیده)

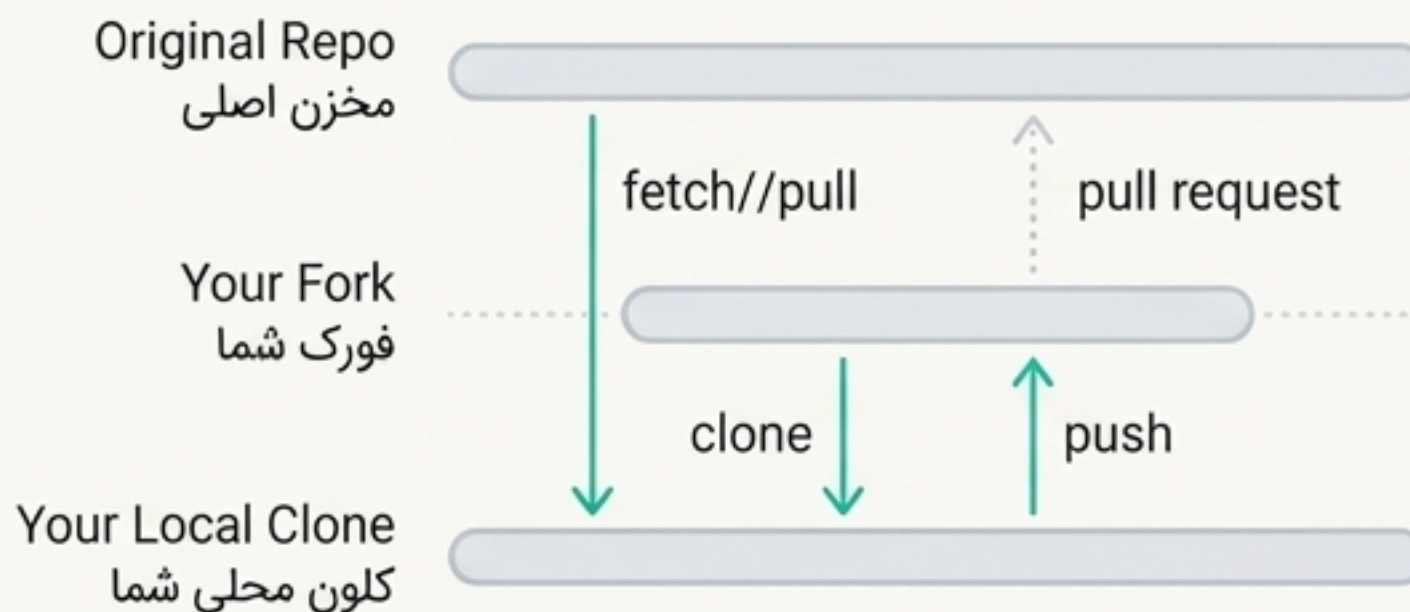
شکارچی باگ و مشارکت در کدباز

این ابزارها برای حل مشکلات خاص طراحی شده‌اند: از پیدا کردن کامیت عامل باگ در یک تاریخچه طولانی تا مدیریت صحیح یک مخزن فورک‌شده.

شکارچی باگ

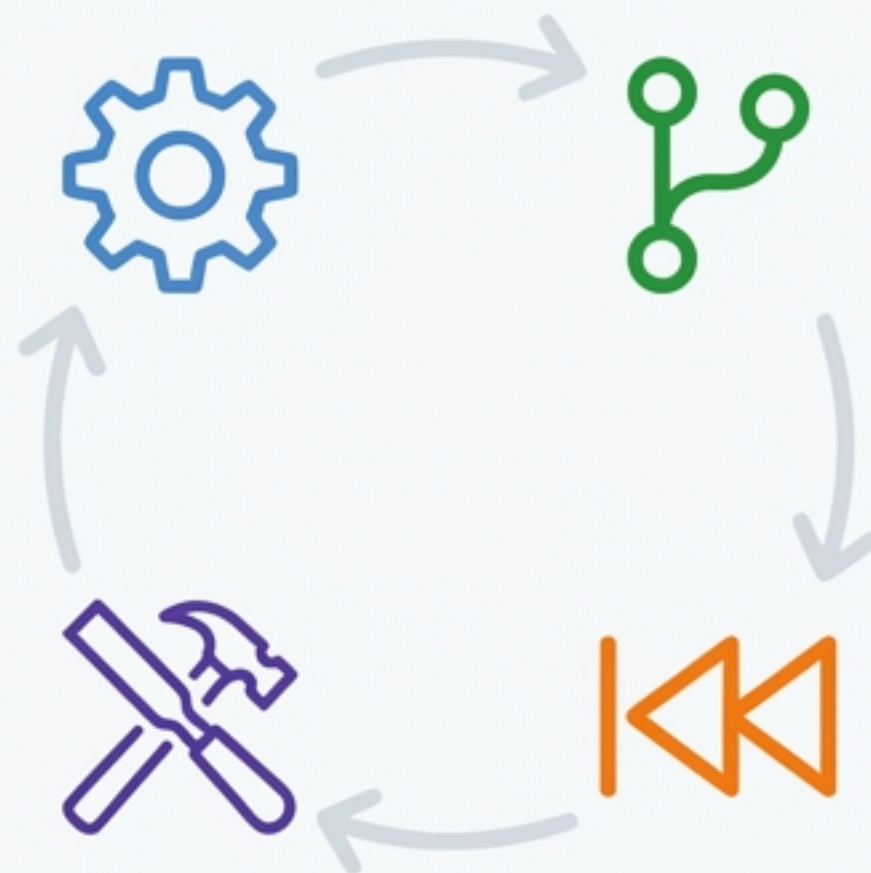


مشارکت در کدباز



- با استفاده از جستجوی دودویی در تاریخچه کامیت‌ها، به شما کمک می‌کند تا به سرعت کامیتی: `git bisect start/bad/good` باعث ایجاد باگ شده را پیدا کنید.
- یک ریموت جدید به نام `upstream` اضافه می‌کند. این دستور برای همگام‌سازی یک: `git remote add upstream [URL]` مغل‌سازی یک مخزن فورک‌شده با مخزن اصلی ضروری است.

جعبه ابزار شما کامل است



شما اکنون ابزارهای لازم برای هر مرحله از چرخه توسعه را در اختیار دارید. از کامیت‌های روزانه گرفته تا بازنویسی پیچیده تاریخچه، می‌توانید با اطمینان کامل در دنیای گیت حرکت کنید.

این دستورات را تمرین کنید. تاریخچه‌ای تمیز و خوانا بسازید. و به طور موثر با تیم خود همکاری کنید.

#توسعه_نرم‌افزار #github #git